

OTA Server 部署手册

面向运维 / K8s 部署同事。本文档覆盖 **Docker Compose 单机部署** 与 **Kubernetes 生产部署说明**（文字版，不含现成 YAML）。

1. 系统概览

OTA Server 由以下组件构成：

组件	说明	是否必须
ota-api	Go HTTP 服务，对外提供管理端 API 与设备端 API	必须
ota-worker	后台 Worker，任务统计快照、超阈值自动回滚	必须
ota-console	React 管理台（生产建议构建静态资源由 Nginx/Ingress 托管）	建议
PostgreSQL	业务数据	必须
Redis	缓存 / 辅助状态	必须
RabbitMQ	消息队列	必须
S3 兼容对象存储	固件包存储（MinIO / AWS S3 / 阿里云 OSS 等）	必须
OIDC Provider	管理台 SSO（Keycloak / 企业 IdP）	生产建议

对外入口（需配置 Ingress / 负载均衡）：

用途	路径前缀	占位符示例
管理端 API	/api/v1/*	https://ota-api.example.com
设备端 API	/device/v1/*	同上（与 API 同域）
管理台 UI	/ 或独立域名	https://ota-console.example.com
健康检查	GET /healthz	https://ota-api.example.com/healthz
固件下载（S3 预签名）	由 check-update 返回	https://s3.example.com/ota-packages/...

占位符说明：下文 `<OTA_API_PUBLIC_URL>`、`<OTA_CONSOLE_PUBLIC_URL>`、`<S3_PUBLIC_BASE_URL>` 等需在部署时替换为实际公网地址。内网 Service 地址与公网地址需分别配置（见第 4 节环境变量）。

2. 前置条件

- Docker Engine 24+ 与 Docker Compose v2（Compose 部署）
- 或 Kubernetes 1.26+ 与 Ingress Controller（K8s 部署）
- 可访问的 PostgreSQL 16+、Redis 7+、RabbitMQ 3+、S3 兼容存储
- TLS 证书（生产环境强烈建议）
- 数据库迁移工具或 Job（见第 6 节）

代码仓库目录：

```
backend/          # Go API + Worker
frontend/         # React 管理台
docker-compose.yml
docker-compose.staging.yml
```

```
.env.example      # 环境变量模板
backend/migrations/  # SQL 迁移脚本 (001 ~ 006)
```

3. Docker Compose 部署

适用于：开发联调、预发验证、单机小规模部署。

当前 `docker-compose.yml` 为开发模式（`go run / npm run dev` 热启动）。生产单机若需长期运行，建议自行构建镜像后替换 `compose` 中的 `image` 与 `command`，或直接使用第 4 节 K8s 思路在单节点跑容器。

3.1 准备环境变量

```
cd /path/to/ota-server
cp .env.example .env
```

编辑 `.env`，至少填写所有 `<...>` 占位符。关键项见第 4 节「环境变量详解」。

本地联调可使用 README 中的默认 Keycloak / MinIO 账号；**生产禁止使用默认密码。**

3.2 启动

```
# 标准启动
docker compose up -d

# 预发叠加（增加 restart: unless-stopped）
docker compose -f docker-compose.yml -f docker-compose.staging.yml up -d
```

3.3 验证

```
docker compose ps
curl -i http://localhost:8080/healthz
# 期望 HTTP 200

# 各服务默认端口（仅开发/内网暴露，生产勿直接暴露数据库端口）
# API:      8080
# Console:  5173
# Postgres: 5432
# Redis:    6379
# RabbitMQ: 5672 / 管理台 15672
# Keycloak: 18080
# MinIO:    9000 / Console 9001
```

3.4 数据库迁移

默认 `API_AUTO_MIGRATE_ON_START=false`，需手动执行迁移：

```
# 方式 A：临时开启自动迁移（仅首次初始化建议）
# .env 中设置 API_AUTO_MIGRATE_ON_START=true 后重启 ota-api

# 方式 B：手动按顺序执行 SQL（推荐生产）
for f in backend/migrations/*.sql; do
```

```
psql "$DATABASE_URL" -f "$f"
done
```

迁移文件顺序：001_init.sql → 002_* → ... → 006_alerts.sql。

3.5 常用运维命令

```
# 查看日志
docker compose logs --tail=200
docker compose logs -f ota-api ota-worker

# 重建
docker compose up -d --build --force-recreate

# 停止
docker compose down

# 停止并清空数据卷（慎用）
docker compose down -v
```

3.6 Compose 服务清单

Service	镜像（当前 dev）	依赖
ota-api	golang:1.22	postgres, redis, rabbitmq
ota-worker	golang:1.22	postgres, redis, rabbitmq
ota-console	node:20	ota-api
postgres	postgres:16	—
redis	redis:7	—
rabbitmq	rabbitmq:3-management	—
keycloak	keycloak:24.0	—
minio	minio	—
minio-init	minio/mc	minio

3.7 生产 Compose（GHCR 镜像）

自有服务由 GitHub Actions 构建并推送到 GHCR，生产环境使用 docker-compose.prod.yml 拉取镜像，不再挂载源码。

镜像命名：

服务	镜像
ota-api	ghcr.io/suge2016/di-ota-api:<tag>
ota-worker	ghcr.io/suge2016/di-ota-worker:<tag>
ota-console	ghcr.io/suge2016/di-ota-console:<tag>

启动步骤：

```
cp .env.prod.example .env
# 合并 .env.example 中的业务密钥到 .env

docker compose -f docker-compose.prod.yml pull
docker compose -f docker-compose.prod.yml up -d
docker compose -f docker-compose.prod.yml ps
curl -fsS http://localhost:8080/healthz
```

镜像版本变量 (`.env`) :

变量	说明	默认
OTA_IMAGE_REGISTRY	registry 前缀 (含 owner)	ghcr.io/suge2016
OTA_IMAGE_TAG	标签	latest

示例: `OTA_IMAGE_TAG=sha-1b1dfc1` 锁定某次 CI 构建。

国内拉取 GHCR (无公司内网 mirror 时建议) :

GHCR 没有官方国内节点, 可在 `.env` 替换 `OTA_IMAGE_REGISTRY` (CI 仍推官方 `ghcr.io` , 仅部署拉取走代理) :

优先级	OTA_IMAGE_REGISTRY	说明
1	ghcr.dockerproxy.com/suge2016	DockerProxy, 将 ghcr.io 换为 ghcr.dockerproxy.com
2	docker.lms.run/ghcr.io/suge2016	1ms 拉取代理, 保留 ghcr.io 路径
3	ghcr.io/suge2016	直连 (海外或网络良好)

验证: `docker pull ${OTA_IMAGE_REGISTRY}/di-ota-api:${OTA_IMAGE_TAG}`

第三方代理非 GitHub 官方, 可用性可能变化; 生产稳定后建议 ACR 同步或自建 pull-through。

Docker Hub 中间件加速 (postgres/redis/minio 等) :

与 GHCR 无关, 在宿主机 `/etc/docker/daemon.json` 配置 `registry-mirrors` (如 `https://docker.lms.run` 、 `https://docker.m.daocloud.io`), 修改后 `sudo systemctl restart docker`。

CI: 工作流见 `.github/workflows/build-images.yml`; push 到 `main` / `feat/**` 或打 `v*` tag 时构建三镜像。PR 仅 build 不 push。

3.8 demogo.work 演示环境 (生产对齐)

`deploy/demogo/docker-compose.demogo.yml` 用于在 **demogo.work** 入口墙下以子路径 `/ota/` 部署, 不启 Keycloak (OIDC 与生产一样为可选)。

项	demogo 配置	生产对齐说明
入口	Caddy handle_path /ota/* → ota-console	子路径需 console 镜像 base: /ota/
编排	/opt/di-ota + docker-compose.demogo.yml	独立 stack, 不占用装箱 DB 5432
设备鉴权	DEVICE_API_AUTH_ENABLED=true	与生产一致, per-device HMAC
管理平台登录	LOCAL_AUTH_ENABLED=true, OIDC_ENABLED=false	OIDC 可选; demogo 用本地账号
公网 API	API_PUBLIC_BASE_URL=https://demogo.work/ota	设备/浏览器经 Caddy 访问

启动概要:

```
# 服务器 /opt/di-ota
cp .env.prod.example .env # 合并 .env.example 业务项
# 必填: OTA_IMAGE_TAG、JWT/DEVICE 密钥、POSTGRES/MINIO 密码
# demogo 关键项:
#   DEVICE_API_AUTH_ENABLED=true
#   LOCAL_AUTH_ENABLED=true
#   OIDC_ENABLED=false
#   API_PUBLIC_BASE_URL=https://demogo.work/ota

docker compose -f docker-compose.demogo.yml up -d
```

设备 secret provision (生产必做, demogo 同样):

```
# 管理员登录后
curl -X PUT "https://demogo.work/ota/api/v1/devices/<SN>/device-secret" \
  -H "Authorization: Bearer <jwt>" \
  -H "Content-Type: application/json" \
  -d '{"device_secret": "<每台唯一 secret>"}'
```

管理台模拟器: `https://demogo.work/ota/#/simulator`, 填写相同 `device_id` 与 `device_secret` 做 HMAC 联调。

4. Kubernetes 部署说明

本节为文字指南, 供自行编写 Manifest / Helm Chart。仓库内暂无现成 K8s YAML 与生产 Dockerfile, 部署时需先构建应用镜像。

4.1 推荐工作负载划分

Deployment	副本	说明
ota-api	≥2 (生产)	无状态, 暴露 HTTP 8080
ota-worker	1~2	无 HTTP, 与 api 共享 DB/Redis/RabbitMQ
ota-console	≥2	静态前端 + Nginx, 或 CDN 托管

不建议放入 Pod 的依赖 (按需选用云服务或集群内独立 StatefulSet):

- PostgreSQL
- Redis
- RabbitMQ
- S3 对象存储
- OIDC (Keycloak 或企业 IdP)

依赖选型由部署同事自行决定: 可用云 RDS / ElastiCache / 托管 MQ / OSS, 也可在集群内 Helm 安装。只需保证 **ota-api** / **ota-worker** 能通过环境变量连上即可。

4.2 镜像构建要点

当前 compose 使用源码挂载 + `go run`, 生产需改为多阶段构建, 示例思路:

```
# ota-api / ota-worker 共用 backend 代码, 入口不同
# api: backend/cmd/api
# worker: backend/cmd/worker

# frontend: npm ci && npm run build → dist/ 由 nginx 镜像托管
```

构建完成后推送到内部镜像仓库，Deployment 引用具体 tag（建议不用 latest）。

4.3 Service 与 Ingress



- 设备端与管理端 API 同域：V9 设备只需配置一个 `<OTA_API_PUBLIC_URL>`。
- **readinessProbe / livenessProbe**：GET `/healthz`，port 8080。
- **S3 下载 URL 必须走公网**：设备从外网拉固件，不能出现 `minio:9000` 等内网 host（见 `S3_PUBLIC_BASE_URL`）。

4.4 ConfigMap 与 Secret 划分

类型	示例键	说明
Secret	JWT_SECRET	管理端 JWT 签名
Secret	DEVICE_SIGNING_SECRET	下载 URL HMAC（与设备 HMAC 分离）
Secret	POSTGRES_PASSWORD	数据库
Secret	S3_ACCESS_KEY_ID / S3_SECRET_ACCESS_KEY	对象存储
Secret	OIDC_CLIENT_SECRET	SSO
ConfigMap	POSTGRES_HOST / REDIS_ADDR / RABBITMQ_URL	连接串
ConfigMap	S3_ENDPOINT	内网 S3 endpoint
ConfigMap	S3_PUBLIC_BASE_URL	公网下载基址
ConfigMap	API_PUBLIC_BASE_URL	公网 API 基址
ConfigMap	OIDC_ISSUER_URL / OIDC_REDIRECT_URL	SSO
ConfigMap	DEVICE_API_AUTH_ENABLED=true	生产开启
ConfigMap	DEVICE_AUTH_TIMESTAMP_TOLERANCE_SEC	默认 300

完整列表见 `.env.example` 与第 4.5 节。

4.5 环境变量详解

复制 `.env.example` 为基准，按环境替换占位符。

4.5.1 公网 URL（必改）

变量	说明	占位符示例
S3_PUBLIC_BASE_URL	设备下载固件的公网基址	<code>https://s3.example.com/ota-packages</code>
API_PUBLIC_BASE_URL	API 公网基址（启用 HMAC 下载代理时）	<code>https://ota-api.example.com</code>
OIDC_REDIRECT_URL	SSO 回调	<code>https://ota-api.example.com/api/v1/auth/sso/callback</code>

`S3_ENDPOINT` 填 **集群内/VPC 内** 地址（如 `http://minio.storage.svc:9000`），与 `S3_PUBLIC_BASE_URL` 分离。

4.5.2 数据库与中间件

变量	说明
POSTGRES_HOST / POSTGRES_PORT / POSTGRES_USER / POSTGRES_PASSWORD / POSTGRES_DB	PostgreSQL
REDIS_ADDR	如 redis.cache.svc:6379
REDIS_PASSWORD	有则填
RABBITMQ_URL	如 amqp://user:pass@rabbitmq.mq.svc:5672/

4.5.3 对象存储

变量	说明
S3_ENDPOINT	内网 endpoint
S3_REGION	区域
S3_BUCKET	桶名，如 ota-packages
S3_ACCESS_KEY_ID / S3_SECRET_ACCESS_KEY	凭证
S3_SIGNED_URL_TTL_SEC	预签名有效期，默认 600 秒

4.5.4 安全与设备鉴权

变量	说明	生产建议
JWT_SECRET	管理端 JWT	随机长字符串
DEVICE_API_AUTH_ENABLED	是否校验设备 HMAC 签名	true
DEVICE_AUTH_TIMESTAMP_TOLERANCE_SEC	签名时间窗（秒）	300
DEVICE_SIGNING_SECRET	下载 URL HMAC（与设备签名分离）	随机长字符串
DEVICE_DOWNLOAD_HMAC_ENABLED	是否走 API 代理下载	默认 false

Provision 设备 secret（公网必做）：

```
PUT /api/v1/devices/<device_id>/device-secret
Authorization: Bearer <admin_jwt>
Content-Type: application/json

{"device_secret": "<产线生成的随机字符串，<=128字符>"}
```

- 在设备注册（CSV 导入）之后、设备首次 OTA 之前执行
- device_secret 存于 t_device.device_secret，勿在设备列表 API 中返回
- 换板 / 返修：重新烧录 secret 并调用此接口覆盖

4.5.5 OIDC（管理平台登录）

变量	说明
OIDC_ENABLED	是否启用 SSO
OIDC_ISSUER_URL	IdP issuer

OIDC_CLIENT_ID / OIDC_CLIENT_SECRET	OAuth 客户端
OIDC_REDIRECT_URL	回调 URL（须与 IdP 注册一致）
OIDC_STATE_SIGNING_KEY	state 签名（可独立，否则回落 JWT_SECRET）
OIDC MOCK_ENABLED	生产设为 false

本地开发 Keycloak realm 见 `deploy/keycloak/realms-ota-dev.json`；生产需在企业 IdP 注册客户端。

4.5.6 其他

变量	说明
API_PORT	默认 8080
API_AUTO_MIGRATE_ON_START	生产建议 false，用 Job 跑迁移
WORKER_TASK_STATS_RETENTION_HOURS	任务统计快照保留，默认 168（7 天）
LOCAL_AUTH_ENABLED	本地账号登录

4.6 部署顺序建议

1. 准备 PostgreSQL / Redis / RabbitMQ / S3 / OIDC
2. 执行数据库迁移（K8s Job 或 CI 流水线）
3. 创建 Secret / ConfigMap
4. 部署 `ota-worker`
5. 部署 `ota-api`，确认 `/healthz` 正常
6. 部署 `ota-console` + Ingress
7. 配置 DNS 与 TLS
8. 冒烟测试：管理台登录 → 上传包 → 创任务 → 设备 `check-update`（见设备对接手册）

4.7 资源与伸缩

组件	参考 requests	说明
ota-api	256Mi / 250m	随 QPS 调整
ota-worker	128Mi / 100m	单副本通常足够
ota-console	64Mi / 50m	静态资源很轻

HPA 可基于 `ota-api` CPU 或自定义指标（Ingress QPS）。

4.8 备份与监控

- **PostgreSQL**：定期备份（任务、设备注册表、升级记录）
- **S3**：桶版本控制 / 跨区域复制（按合规要求）
- **日志**：采集 `ota-api` / `ota-worker` stdout
- **告警**：Ingress 5xx、`/healthz` 失败、Worker 异常退出

5. 外部依赖选型说明

部署同事可按 infra 规范自行选择，以下为常见组合：

依赖	集群内自建	云服务示例
PostgreSQL	StatefulSet / Helm postgresql	AWS RDS、阿里云 RDS

Redis	Helm redis	ElastiCache、阿里云 Redis
RabbitMQ	Helm rabbitmq	Amazon MQ、CloudAMQP
S3	MinIO StatefulSet	AWS S3、阿里云 OSS、腾讯云 COS
OIDC	Keycloak	企业 Azure AD / Okta / 自建 IdP

连接配置原则：

- API / Worker 使用 **内网地址** 访问 DB、Redis、MQ、S3 endpoint
- 设备与浏览器使用 **公网 URL** (`S3_PUBLIC_BASE_URL` 、Ingress 域名)
- 防火墙：设备仅需访问 `<OTA_API_PUBLIC_URL>` 与 `<S3_PUBLIC_BASE_URL>` 域名

6. 数据库迁移

迁移脚本位于 `backend/migrations/`，**必须按文件名顺序执行**：

文件	内容概要
001_init.sql	初始表结构
002_enhance.sql / 002_task_stats_snapshot_idx.sql	增强与索引
003_task_and_package_state.sql	任务与包状态
004_user_management.sql	用户管理
005_architecture_v2.sql	v2 设备注册表、任务快照等
006_alerts.sql	告警
007_device_secret.sql	设备 per-device secret 列

K8s 可封装为 `Job`，在 `api` 启动前跑完；失败则阻止发布。

7. 生产安全检查清单

- ☐ 所有默认密码已更换 (Postgres、MinIO、Keycloak、RabbitMQ guest 等)
- ☐ `DEVICE_API_AUTH_ENABLED=true`
- ☐ 每台设备已 `PUT .../device-secret provision`
- ☐ Ingress 限流与签名失败告警已配置
- ☐ `OIDC MOCK_ENABLED=false`；若未接 IdP 则 `OIDC_ENABLED=false` 且 `LOCAL_AUTH_ENABLED=true`
- ☐ 若已接企业 IdP：`OIDC_ENABLED=true`，`LOCAL_AUTH_ENABLED=false` (或保留 `break-glass` 本地账号)
- ☐ `S3_PUBLIC_BASE_URL` 为设备可达的 HTTPS 地址
- ☐ Ingress 启用 TLS
- ☐ 数据库、Redis、RabbitMQ 不对公网暴露
- ☐ Secret 通过 K8s Secret / 密钥管理系统注入，不写入镜像
- ☐ `API_AUTO_MIGRATE_ON_START=false`，迁移由受控 Job 执行

8. 常见问题

Q：`check-update` 返回的 `download_url` 含内网 host (如 `minio:9000`)

A：检查 `S3_PUBLIC_BASE_URL` 是否配置为公网地址；API 进程需能访问 `S3_ENDPOINT` 生成预签名 URL。

Q：管理平台 SSO 回调 404

A：核对 `OIDC_REDIRECT_URL` 与 IdP 注册 redirect URI 完全一致（含 https）。

Q：设备报 401

A：检查 `Authorization: Device ...` 签名：path 须为 `/device/v1/...`、timestamp 在容忍窗内、`device_secret` 与平台一致。

Q：设备报 2007

A：该 SN 未 provision secret，调用 `PUT /api/v1/devices/:id/device-secret`。

Q：Worker 不跑统计 / 不回滚

A：检查 `RABBITMQ_URL`、Worker Pod 日志、Postgres 连通性。

Q：Compose 与 K8s 环境变量不一致

A：以 `.env.example` 为单一事实来源，Compose 用 `.env`，K8s 用 ConfigMap/Secret 映射同名键。

9. 相关文档

- 仓库 README：快速启动与 API 列表
- 设备对接：doc/device-integration-v9.md
- 架构 v2：doc/架构改进方案_v2_2026-06-05.md